Práctica 04 - README

Equipo: Los Troyanos
Integrantes:

- Huaxcuahutli Teopantzin José Luis (320204603)
- Martínez Martínez Marco Antonio (423116344)
- Santos Alcocer Antonio de Jesús (420003090)

Notas sobre la práctica:
Durante la realización de esta práctica se usaron 2 patrones de diseño, los cuales son **Composite** y **Adapter**. Cada uno con un propósito dentro de la práctica para el correcto funcionamiento y optimización de esta, el modo en que se implementaron cada uno son los siguientes.

La implementación del patrón de diseño Composite está reflejada en el paquete **rockBuster**, y este tiene como fin tratar de manera uniforme tanto a productos simples como a colecciones complejas de obras. Para lograr esto, el paquete cuenta con la interfaz **Producto**. Esta interfaz es implementada por la clase **Pelicula**, que actúa como la hoja simple (Leaf), y por la clase **Saga**, que funge como el nodo compuesto (Composite). Esto permite que una **Saga** contenga una lista dinámica de componentes de tipo Producto, permitiendo anidar tanto películas individuales como otras subsagas. Finalmente, otra gran ventaja es que operaciones complejas, tales como el cálculo de la duración acumulada y el cálculo del precio con el 5% de descuento, se resuelven de forma recursiva a lo largo de toda la jerarquía sin que el catálogo requiera distinguir el tipo concreto de elemento.

La implementación del patrón de diseño **Adapter** está reflejada en el paquete **adaptadores**, y este tiene como fin integrar un subsistema externo de música preexistente al catálogo de la tienda sin alterar su diseño original. Para lograr esto, el paquete cuenta con la clase **AdaptadorDiscoAProducto**, que implementa la interfaz **Producto** y mantiene internamente una referencia por composición a un objeto **Disco** del paquete **mixdown**. La clase adapta la interfaz incompatible de la tienda de discos hacia las operaciones esperadas por el sistema de RockBuster. Finalmente, esto permite que los discos musicales puedan ser catalogados, mostrados, rentados y consultados dentro de la tienda exactamente bajo el mismo contrato de cualquier producto multimedia.

El manejo de errores en operaciones críticas se encuentra estructurado en el paquete **excepciones**, el cual contiene las excepciones verificadas *SinProductoExcepcion*, *NoExisteProductoExcepcion* y *SinDiscosExcepcion*, asegurando un control robusto ante la ausencia de existencias o productos no localizados.

Finalmente, el filtrado y búsqueda de productos están gestionados en el paquete **filtro**, y estos tienen como fin proporcionar mecanismos dinámicos y desacoplados para consultar el catálogo bajo diversos criterios sin alterar su estructura interna. Para lograr esto, el paquete cuenta con la interfaz **FiltroProducto**, encargada de definir el método *cumpleCriterio(Producto producto)*, el cual es implementado en las clases **FiltroGenero** y **FiltroPrecioMaximo**. Esto

permite que el catálogo evalúe de manera uniforme tanto películas simples como sagas compuestas y discos adaptados, ya sea comparando el género sin distinción de mayúsculas o validando un costo tope de renta. Finalmente, la aplicación de estos filtros se refleja a través del método f*iltrar(FiltroProducto filtro)* en la clase **Catalogo**, devolviendo un subconjunto de productos que satisfacen la condición especificada para su posterior despliegue estructurado en pantalla.

Comandos de compilación:
Compilar en Linux:
Se necesita estar en la dirección: *src/main*

- Para compilar con javac
  $ javac -d bin $(find . -name "*.java")
- Ejecutar en Linux:
  $ java -cp bin Main

Compilar en WIndows (Cmd)
Primero se necesita ir a la dirección base de la práctica

- Compilar
  $ dir /s /b src*.java > sources.txt
  $ javac -d bin @sources.txt
- Ejecutar
  $ java -cp bin Main